

Práctica 5: Depuración y Análisis Estático de Código

Verificación y Validación Grupo 215

Imad Habib Jali (901421)
Jorge Bellido Lobera (903080)

1. Métodos de prueba en InventoryTest no diseñados en pizarra

Tras analizar la clase `InventoryTest.java` original en el módulo `GildedRose0`, se observa que, aunque la mayoría de los escenarios tienen correspondencia con el Excel, la estructura del código es más granular:

- **Fragmentación por atributo:** Mientras que en el diseño de pizarra (Excel) una única prueba (ej. P9) define resultados para `sellIn` y `quality`, el código original separa estas validaciones en métodos independientes:
`should_lower_the_sellIn_by_one_for_normal_items` y
`should_lower_the_quality_by_one_for_normal_items`.
- **Tests de Límite Mínimo:** El test `should_not_lower_the_quality_below_zero` (P9b) ya estaba implementado originalmente para asegurar que la calidad de los artículos normales nunca sea negativa, un caso de borde fundamental no siempre detallado en el diseño inicial.

2. Métodos diseñados en pizarra que no aparecen en InventoryTest

Al contrastar la clase original con la hoja de cálculo `pruebas-kata-Gilded-Rose.xlsx`, se detectaron ausencias significativas en la batería de pruebas inicial:

- **Aged Brie (P1, P1b, P2, P2b, P2c):** No existían pruebas para el aumento de calidad de este artículo, ni para su comportamiento tras la fecha de venta (P2*).
- **Backstage Passes (P4 a P8):** Faltaba toda la lógica compleja de incrementos de calidad (+1, +2, +3) según los días, límite 50 y la caída a 0 tras el concierto.
- **Artículos Normales Expirados (P10, P10b, P10c):** No se validaba el decrecimiento doble de calidad una vez que `sellIn < 0`.
- **Valores Límite de Frontera:** Casos específicos para `sellIn = 0` (P2d, P5d, P10d) no estaban cubiertos originalmente.

3. Pruebas añadidas para lograr una cobertura del 100 %

Tras completar los casos del Excel, la cobertura de ramas se situó en un 91 %. Se añadieron pruebas específicas para cubrir las condiciones de calidad máxima "Sulfuras en bloque caducado". A continuación se muestra el código final de las pruebas añadidas:

```
1 // Prueba P2d: Aged Brie en el limite de caducidad (sellIn=0) aumenta +2
2 @Test
3 public void
4     should_increase_the_quality_of_aged_brie_twice_as_fast_once_the_sell
5     _in_date_has_passed() {
6     Item agedBrie = new Item("Aged Brie", 0, 25);
7     Inventory inventory = createInventory(agedBrie);
8     inventory.updateQuality();
9     assertEquals(27, agedBrie.getQuality());
10 }
11 // Prueba P10c: Artículo normal caducado con calidad 0 (Cobertura linea
12 49)
13 @Test
14 public void should_not_lower_the_quality_below_zero_when_expired() {
15     Item normalItem = new Item("+5 Dexterity Vest", 0, 0);
16     Inventory inventory = createInventory(normalItem);
17     inventory.updateQuality();
18     assertEquals(0, normalItem.getQuality());
19 }
20 // Prueba P3 variante: Sulfuras con sellIn negativo (Cobertura linea 50)
21 @Test
22 public void should_never_changes_quailty_of_Sulfuras_when_expired() {
23     Item sulfuras = new Item("Sulfuras, Hand of Ragnaros", -1, 80);
24     Inventory inventory = createInventory(sulfuras);
25     inventory.updateQuality();
26     assertEquals(80, sulfuras.getQuality());
27 }
28
29 // Prueba P2b: Aged Brie caducado con calidad al maximo (Cobertura linea
30 59)
31 @Test
32 public void
33     should_not_increase_the_quality_of_aged_brie_over_50_when_expired() {
34     Item agedBrie = new Item("Aged Brie", 0, 50);
35     Inventory inventory = createInventory(agedBrie);
36     inventory.updateQuality();
37     assertEquals(50, agedBrie.getQuality());
38 }
```

4. Descripción de los defectos de los módulos GildedRose1 y GildedRose2

Defecto en GildedRose1

- **Localización:** `Inventory.java`, línea 27.
- **Descripción:** Error de límite (Off-by-one). Se usaba `<= 50` en lugar de `<50` al incrementar la calidad de los *Backstage passes*.
- **Consecuencia:** Si un pase tenía calidad 49 y faltaban menos de 11 días, el primer incremento lo subía a 50, y la condición defectuosa permitía un segundo incremento ilegal a 51.

Defecto en GildedRose2

- **Localización:** `Inventory.java`, línea 17.
- **Descripción:** Inversión lógica. Se usaba el operador `==` para *Sulfuras* en una cláusula que requería `!=`.
- **Consecuencia:** Se invertía el comportamiento del sistema: los artículos normales no perdían calidad, mientras que a *Sulfuras* se le restaba calidad erróneamente.

5. Violaciones del inspector (GildedRose0) y valoración crítica

Análisis Base (12 Warnings)

Se detectaron violaciones críticas para la robustez del código:

- **Comparación de Strings con `==`:** Presente en 8 puntos de `Inventory.java`. Es un riesgo severo de estabilidad si los datos dejaran de ser constantes literales.
- **Bucle For optimizable:** Sugerencia de usar *for-each* en la línea 14 para mejorar la legibilidad.
- **Lógica redundante:** La expresión `quality - quality` en la línea 55 es innecesaria y debe simplificarse a `setQuality(0)`.

Análisis Extremo (446 Warnings)

Al activar todas las reglas de Java, las alertas subieron de 12 a 446. Los ejemplos más absurdos incluyen:

- **Naming Conventions:** Alertas masivas contra los tests por usar guiones bajos, ignorando su propósito de legibilidad en Testing.
- **Javadoc:** Exigencia de comentarios en métodos triviales como `getQuality()`, generando ruido y mantenimiento innecesario.

Opinión Crítica: El experimento demuestra que la calidad no se logra mediante la acumulación de reglas, sino mediante su selección. El ruido masivo del análisis extremo provoca "fatiga de alertas", ocultando errores críticos entre cientos de avisos estilísticos insignificantes.